

UNIVERSITÄT
LEIPZIG

Tutorial 2

Basic Programming Paradigms

Leipzig, 29.10.2026

Enny May

Outline

- Solution programming exercise
- Questions from last time
- Repetition of lecture content
- Questions

Solution to programming task

Exercise 2 – Standard Output (5 minutes)

- Write a program that outputs your name, your student number and course of study.
- Create a variable that stores your first and last name and use it for outputting your name.

Notes

- Use following include directive in order to use strings: *#include <string>*

Exercise 2 – possible solution

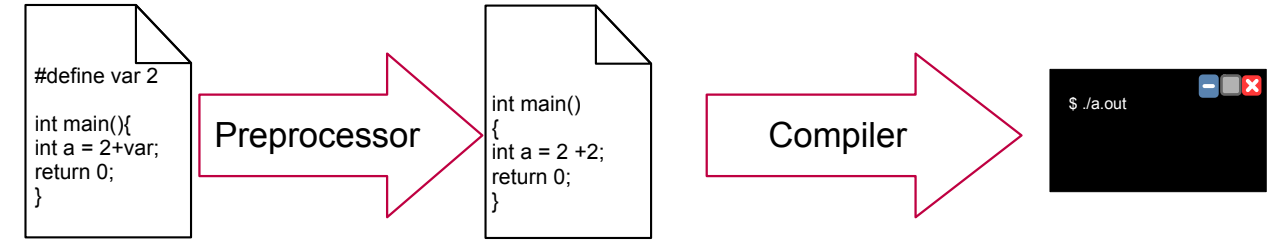
```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    string name{"Vorname Nachname"};
    cout << "Name: " << name << endl;
    cout << "Matrikelnummer: " << 1111111 << endl;
    cout << "Studiengang: Wirtschaftsinformatik" << endl;

    return 0;
}
```

How does compilation work?



Questions

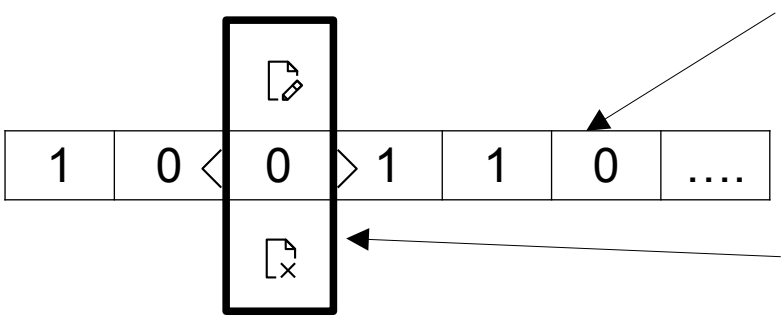
The professor uses g++ -std=c++23 file.cpp. Why?

g++ -std=c++23 file.cpp -o newName.out

- *-std=c++23* explicitly specifies the C++ version used to compile
- *-o newName.out* names the output file produced by the compiler like specified

Universal Turing Machine

- Can compute every computable function
- Programming instructions:
  - Read
  - Write
  - Move
- Read-write head moves across the tape and follows the algorithm specified there



Endless tape with encoded data → instructions

Movable read-write head

Why can't a Turing machine actually be built?

- Requires endless tape
- Could take endless time processing the tape

Repetition of lecture content

Comments

- Not processed program text
- Preprocessor removes any comment
- Cannot be nested
- Information for the programmer
- Other programs are able to process special comments

```
1  //comment (only one line)
2
3  /* comment */ <no comment>
4
5  /* comment-
6     over
7     several
8     lines */
```

Libraries

- Does not extent the programming language
- Provides new functionality
- *#include <library>*

Static library

- Directly integrated into the executable file
  - Larger executable file
  - Self-sufficient during runtime

Dynamic library

- Linked during runtime
  - Smaller executable file
  - More efficient use of system resources

Namespaces

- Scope used for the declaration of functions, variables, types,...
- Prevents name collisions
- Used for elements of libraries
- Same namespace can be used several times

access

- *using namespace <namespace>;*
  - Imports whole namespace
  - Can lead to name collisions
- Access operator „::“
  - e.g.: *std::cout*

main-function

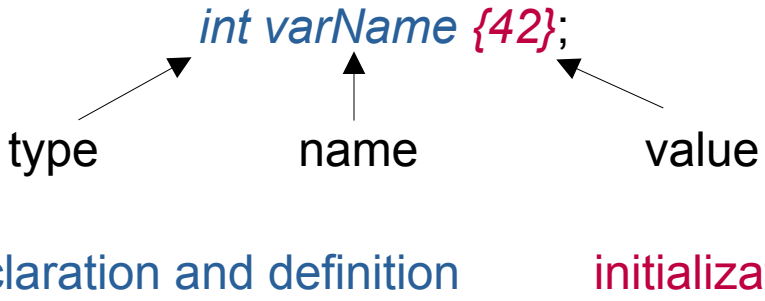
- Entrance point of the program
- Possible signatures:
  - *int main();* ~ no arguments are passed
  - *int main(int argc, char* argv);* ~ q.v. APP
- Return value is relevant for the operating system
  - *return 0* ~ program was successfully executed
  - Every other value: some error occured during execution

Variables

- Typed memory location that usually has a name

Naming scheme

- First character: alphabet character or underscore
- Following characters: alphanumberic character or underscore



Declaration / Definition / Initialization

Declaration

- Introduces name of the variable or object
- Provides signature for the compiler

Definition

- Allocates memory without assigning a value

Initialization

- Assigns a value

Overview of important data types

Data type	**Declaration of a variable**	**explanation**
Integer	**int** number {42};	- for integral numbers e.g. -1, 2, 100
Double	**double** number {42.42};	- for floating point numbers e.g. -1.1, 1.01101
Character	**char** character {'a'};	- comprises ASCII table - alphanumeric characters have to be enclosed in single quotes

Overview of important data types

Data type	**Declaration of a variable**	**explanation**
Boolean	**bool** boolean {true};	- only two possible values: *true* and *false*
String	**string** words {“Hello World“};	- not a fundamental data type → *#include <string>* - represents a sequence of characters - needs to be enclosed in double quotes

Output Streams

- Flow of data out of a program

Buffered output

- Stored output, until printing is forced
  - *cout* (standard output stream)
  - *clog* (error stream)
- advantage: more resource efficient
- disadvantage: no output if program crashes

Ungebufferter Output

- Immediately printed in the console
  - *cerr* (standard error stream)

Flushing the buffer

endl

- Stream manipulator
- Empties buffer and starts a new line

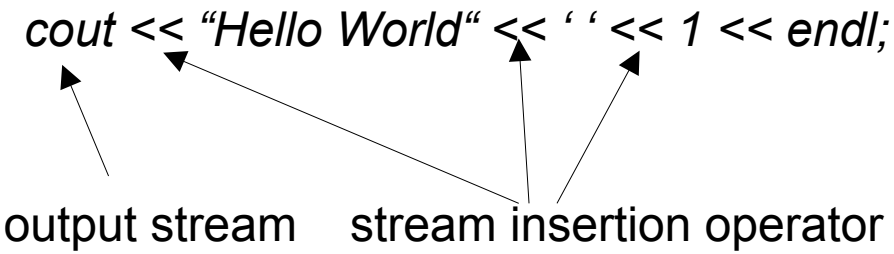
flush

- Stream manipulator
- Empties buffer

cin

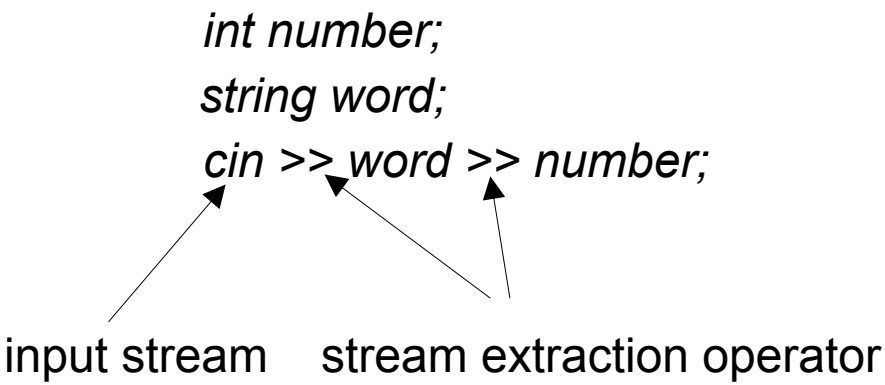
- Empties buffer with a minimal loss of performance

Standard Output Stream



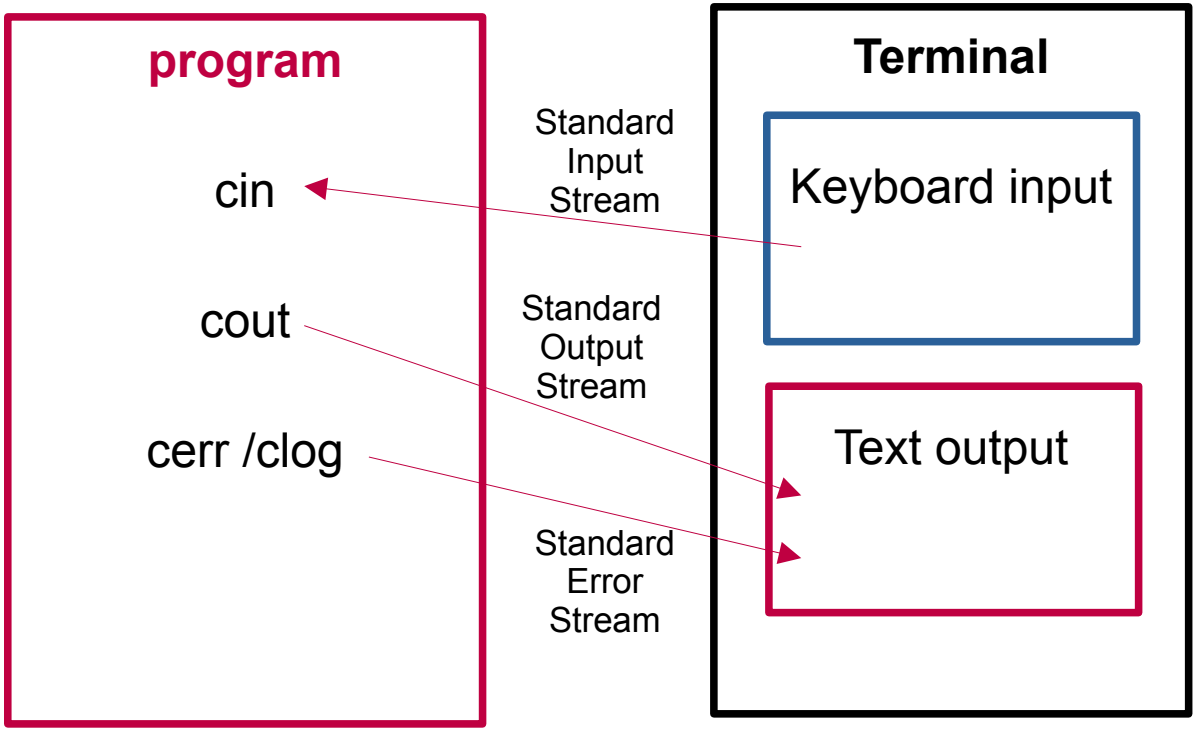
- Stream insertion operators are used between literals of different data types, variables and stream manipulators
- *cout* sends a stream of characters to the terminal

Standard Input Stream



- Stream extraction operator is used between different variables
  - Converts extracted characters into values of the required type
- Gets a stream of characters from the console window

Connection between stream and console



Code example for input and output streams

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(){
6       int number;
7
8       cout << "Please input a number: ";
9       cin >> number;
10
11      cout << "Thank you for inputting the number "-
12           << number << '.' <<endl;
13  }
```

Escape sequence

- To represent characters that have a special effect or meaning

important escape sequences

- Single quote: \‘
- Double quote: \“
- Question mark: \?
- Backslash: \\
- Linefeed: \n
- horizontal tab: \t

Boolean data type

- Type for logical values (true and false)
- Needed for conditional expressions in control structures

Automatic conversions from bool to integer

- True ⟶ 1
- False ⟶ 0

Automatic conversions from integer to bool

- !=0 ⟶ True (e.g. bool b = -1; is true)
- 0 ⟶ False

Relational operators

- Result is either true or false (boolean value)
- `<` smaller than
- `<=` smaller than or equal to
- `>` larger than
- `>=` larger than or equal to
- `==` equal to
- `!=` not equal to

Combination of logical expressions

Logical conjunction

- C++ syntax: A && B
- Mathematical notation: A $\wedge$ B

A&&B	B == 1	B == 0
A == 1	true	false
A == 0	false	false

Logical disjunction

- C++ syntax: A || B
- Mathematical notation: A $\vee$ B

A || B	B == 1	B == 0
A == 1	true	true
A == 0	true	false

Code example for logical expressions

```cpp
#include <iostream>

using namespace std;

int main(){
    cout << boolalpha //stream manipulator that-
                      //ensures that boolean values-
                      //are outputed either as "true" or "false"
         << "(5 < 4) is " << (5 < 4) << '\n'-
         << "(5 == 4) is " << (5 == 4) << '\n'
         << "(5 != 4) is " << (5 != 4) << '\n'
         << "(0 && 1) is " << (0 && 1) << '\n'
         << "(0 || 1) is " << (0 || 1) << '\n'
         << "(5 && 4) is " << (5 && 4) << endl;
}
```

Control structures

If-else-statement

```
if(condition){
    //do something
}
else{
    //do something
}
```

- Depending on a condition some code is executed or not
- If condition is true: execution of the if-branch
- Else: execution of the else-branch

Code example for if-statements

```cpp
#include <iostream>

using namespace std;

int main(){
    bool condition1 {true && false};
    bool condition2 {5>2};

    if(condition1){
        cout << "This should never happen." <<endl;
    }
    else{
        cout << "Condition1 is false." <<endl;
    --  --
        if(condition2){
            cout << "Condition2 is true." <<endl;
        }
        else{
            cout << "This should never happen." <<endl;
        }
    }
```

Control structures

while-loop

```
while(condition){
    //do something
}
```

- Checks condition first
- While condition is true: execution of the loop

Code example for while-loops

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(){
6       int i {1};
7       while(i < 9){
8           if(i%2 == 0){
9               cout << i << ' ';-
10          }
11          i = i+1;
12      }
13  }
```

i	Output
1	
2	
3	
4	
5	
6	
7	
8	
9	

Control structures

do-while-loop

```
do{
    //do something
}while(condition);
```

- Executed at least once
- Checks condition after first execution
- While condition is true: further execution of the loop

Code example for do-while-loops

```cpp
#include <iostream>

using namespace std;

int main(){
    int i {};

    do {
        cout << i <<endl;
        i = i+1;
    } while(i < 0);
}
```

i	Output
0	
1	

Control structures

for-loop

```
for(variable; condition; increment){
    //do something
}
```

- Initializes counting variable once
- Before first execution: checks condition
- After each execution:
  - increments the variable
  - Checks condition before executing the loop

Code example for for-loops

```cpp
#include <iostream>

using namespace std;

int main(){
    for(int i {0}; i<7; ++i){
        if(i%2){
            cout << i << ' ';
        }
    }
}
```

i	Output
0	
1	
2	
3	
4	
5	
6	
7	

Break and continue

Break-statement

- Stops execution/ jumps out of a loop

Continue-statement

- Stops current iteration of the loop
- Continues with the next iteration, if the condition is still fullfilled

Code example break and continue

```cpp
#include <iostream>

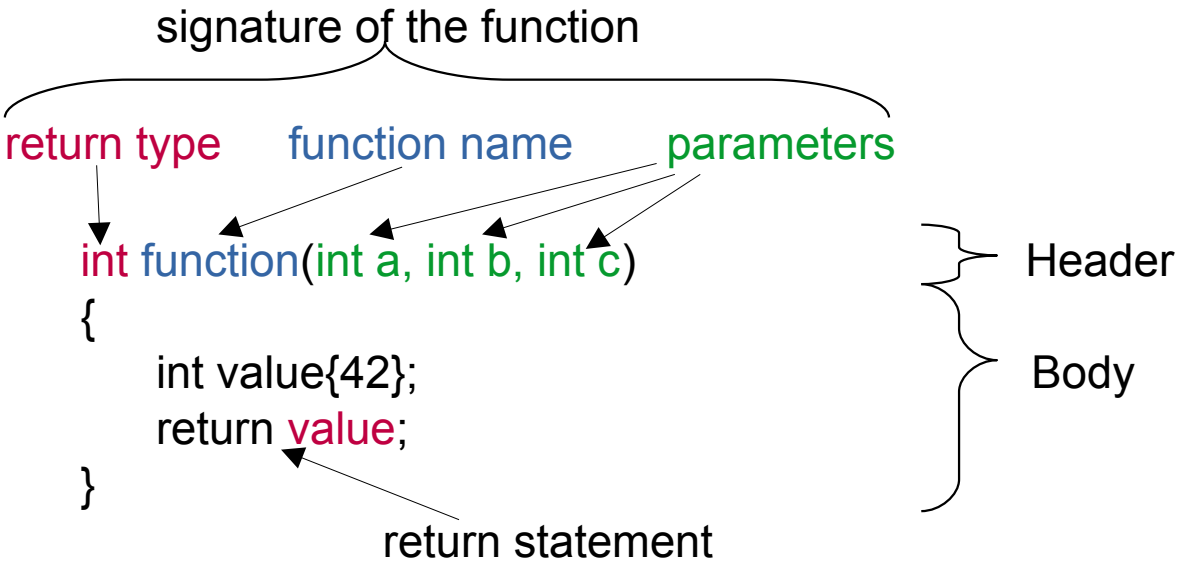
using namespace std;

int main()
{
    for(int i{}; i<10; ++i){
        if(i==1){
            continue;
        }
        cout << i <<endl;
        if(i == 2){
            break;
        }
    }
}
```

i	Output
0	
1	
2	

Functions

- Allow reuse of statements
  - Mean to prevent redundant code



Return type of functions

- Data type of the value returned from a function

Return type void

- Function returns no value
- Corresponds to a procedure in the imperative programming paradigm

Other return types

- Function returns exactly one value
- Corresponds to a function in the imperative programming paradigm

Call by value

- Parameter is copied before entering the function
  - Only copy can be changed within the function
  - No effect on the actual parameter
- Parameters are local variables within the function
  - Destroyed after function is exited
- Order of the parameters is important

```cpp
#include <iostream>

int function(int a)
{
    a = a+1;
    return a;
}

int main()
{
    int a = 1;
    std::cout << "a of the function: " << function(a) << std::endl;
    std::cout << "a of the main-function: " << a << std::endl;
}
```

Output

a of the function: 2

a of the main-function: 1

→ the *a* used in the function is only a copy of the *a* in the main-function

Code example for functions

```
1   #include <iostream>
2
3   using namespace std;
4
5   int greaterNumber(int a, int b){
6       if(a > b)
7           return a;
8       else-
9           return b;
10  }-
11
12  int main(){
13      cout << greaterNumber(5,8) <<endl;
14  }
```

Questions?

Feedback



particify.uni-leipzig.de

9261 7839





UNIVERSITÄT
LEIPZIG

THANK YOU!

Enny May
Professur für Wirtschaftsinformatik,
insbesondere Softwareentwicklung
für Wirtschaft und Verwaltung

e.may@studserv.uni-leipzig.de